



Data Science

Program Studi Informatika

Sesi 11 – Unsupervised Learning: Clustering (K-Means & Hierarchical)

Syahid Abdullah, S.Si, M.Kom



Outline



Pembukaan & Motivasi

Apa itu Clustering? Posisi clustering dalam unsupervised learning: dari data ke segmentasi



Lanskap Clustering

Partitional vs Hierarchical · Jenis algoritma clustering · Use case



K-Means Clustering

Fungsi objektif WCSS, algoritma iteratif, Hierarchical Clustering



Metode Elbow

WCSS/inertia, cara membaca grafik, menentukan K optimal



Unsupervised Learning

Clustering · Segmentasi Data · K-Means · Hierarchical · Algoritma Lainnya



K-Means vs Hierarchical Clustering

K-Means Clustering

Partitional clustering: membagi data ke dalam K cluster berdasarkan jarak ke centroid terdekat.

Analogi: menentukan K titik pusat, lalu kelompokkan tetangga terdekat

Task:

Assignment Step

Setiap titik data di-assign ke centroid terdekat
Berdasarkan jarak Euclidean terkecil

Update Step

Centroid diperbarui = rata-rata anggota cluster
Diulang hingga centroid konvergen

Karakteristik: cocok untuk cluster berbentuk bulat (spherical), dataset besar, K sudah diketahui

Hierarchical Clustering

Membangun hierarki cluster secara bertingkat tanpa perlu menentukan K di awal.

Analogi: menyatukan kelompok kecil menjadi kelompok besar secara bertahap (seperti pohon keluarga)

Task:

Agglomerative (Bottom-Up)

Mulai dari tiap titik sebagai cluster sendiri
Digabung bertahap, paling umum digunakan

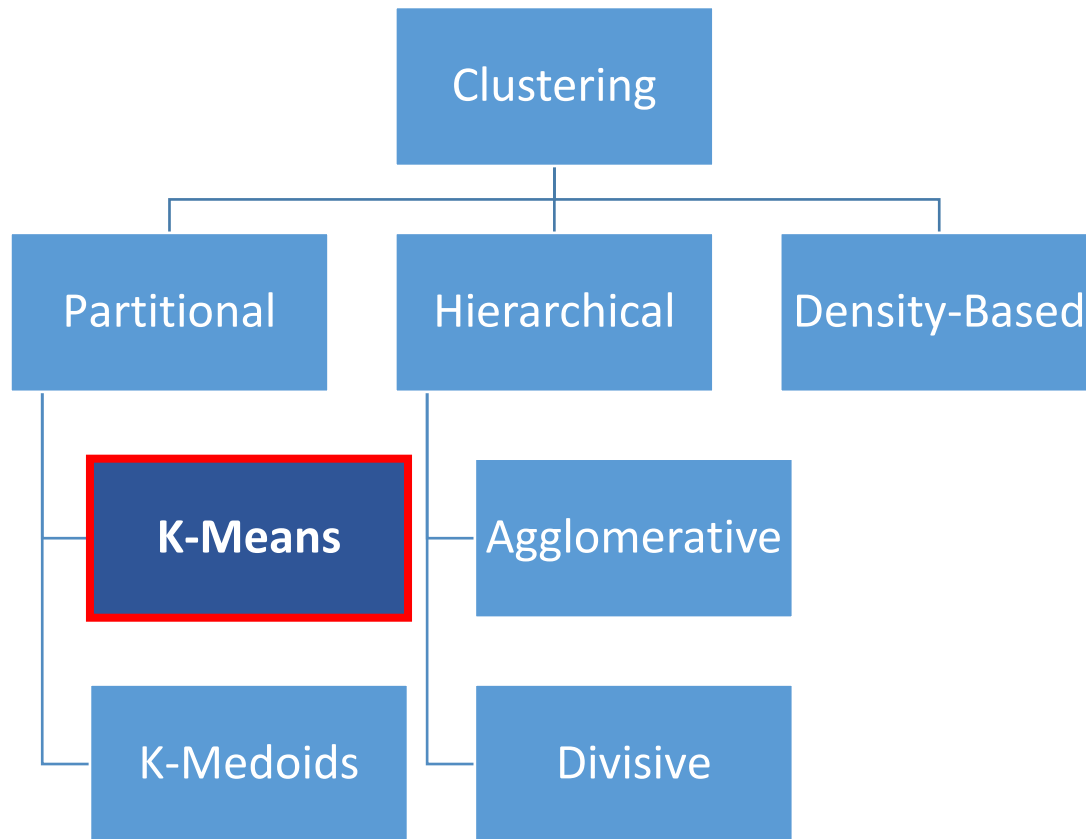
Divisive (Top-Down)

Mulai dari satu cluster besar, dipecah bertahap
Jarang dipakai, komputasi lebih mahal

Karakteristik : cocok untuk dataset kecil-menengah, visualisasi dendrogram, K tidak perlu ditentukan di awal



Jenis Algoritma Clustering



Jenis Algoritma Clustering:

K-Means Clustering

Partitional, berbasis centroid
Cepat, skalabel untuk data besar

Hierarchical Clustering

Bertingkat, hasilkan dendrogram
Tidak perlu tentukan K di awal

DBSCAN

Density-based clustering
Tahan noise & outlier, bentuk cluster bebas

Gaussian Mixture (GMM)

Model probabilistik (soft clustering)
Setiap titik punya probabilitas keanggotaan



K-Means Clustering

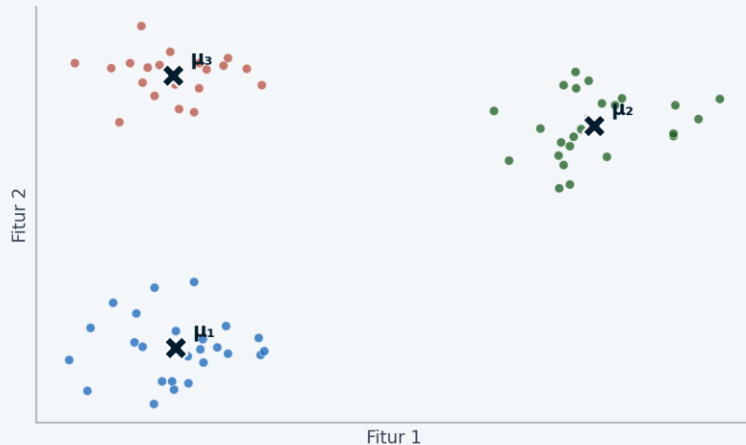
Fungsi Objektif (WCSS) · Algoritma Iteratif · Inisialisasi & Konvergensi



Konsep K-Means Clustering

Ide Dasar

K-Means mencari K titik pusat (centroid) yang meminimalkan total jarak kuadrat antara setiap titik data dengan centroid clusternya.



Kapan digunakan:

- Data tidak berlabel dan ingin mengelompokkan berdasarkan kemiripan
- Cluster berbentuk bulat (spherical) dan jumlah cluster K sudah diperkirakan
- Butuh algoritma yang cepat dan skalabel untuk dataset besar
- Baseline sebelum mencoba clustering lebih kompleks (DBSCAN, GMM)

Fungsi Objektif K-Means (WCSS)

$$WCSS = \sum ||x - \mu_k||^2$$

J

Nilai WCSS yang diminimalkan

K

Jumlah cluster yang ditentukan di awal

μ_k

Centroid (titik pusat) dari cluster ke-k

x

Titik data individual dalam cluster

Contoh:

$K = 3$, $WCSS \text{ awal} = 245.8 \rightarrow \text{setelah iterasi konvergen, } WCSS = 82.3$
(cluster makin rapat)

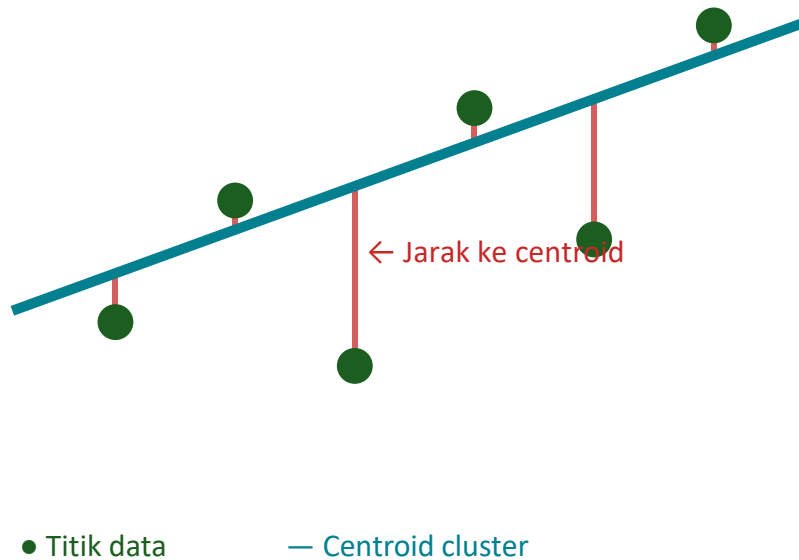


Fungsi Objektif

Jarak: Titik Data ke Centroid

Untuk setiap titik data x , jarak kuadrat ke centroid cluster μ diukur menggunakan Euclidean distance:

$$d(x, \mu) = \|x - \mu\|^2$$



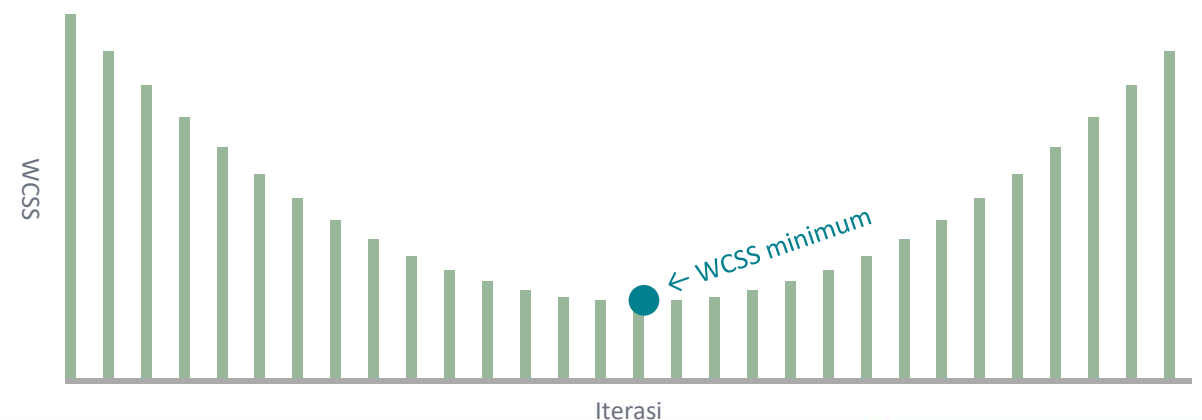
Fungsi Objektif K-Means: WCSS

$$WCSS = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2$$

Mengapa dikuadratkan?

- Memastikan jarak selalu bernilai positif (tidak saling menghapus)
- Penalti lebih besar untuk titik yang jauh dari centroid
- Mendorong cluster yang lebih rapat dan kompak

Tujuan: Minimumkan WCSS





Algoritma K-Means

Algoritma Iteratif K-Means

Analogi: mengelompokkan orang berdasarkan siapa yang paling dekat, pindah ke titik tengah kelompok, ulangi.

1

Inisialisasi Centroid

Pilih K titik centroid awal secara acak dari data

2

Assignment Step

Assign setiap titik data ke centroid terdekat

3

Update Step

μ_k := rata-rata semua titik dalam cluster k

4

Ulangi hingga Konvergen

Iterasi sampai centroid tidak berubah (WCSS stabil)

K = jumlah cluster: ditentukan sebelum algoritma dijalankan (lihat Metode Elbow)

Assignment & Update Rule

$$c_i := \operatorname{argmin}_k \|x_i - \mu_k\|^2$$
$$\mu_k := \operatorname{mean}(x_i : c_i = k)$$

Dampak Pemilihan Jumlah Cluster (K)

Terlalu Kecil

- ✓ Model sederhana, cepat dihitung
- ⚠ Underfitting: cluster gabungan tidak representatif

Tepat

- ✓ Cluster merepresentasikan struktur data
- ⚠ Perlu tuning: gunakan metode Elbow atau Silhouette

Terlalu Besar

- ✓ WCSS sangat kecil (hampir 0)
- ⚠ Overfitting: cluster terlalu granular, sulit diinterpretasi



```
from sklearn.cluster import KMeans
# scikit-learn menggunakan algoritma Lloyd
# (assignment + update, iteratif hingga konvergen)
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)
# centroid = model.cluster_centers_
# label = model.labels_
```



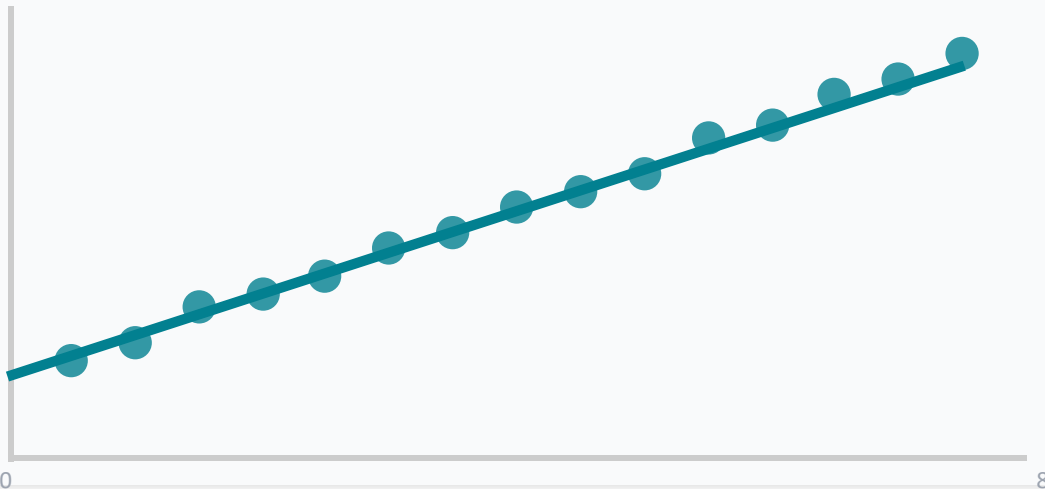
Agglomerative vs Linkage Methods

Agglomerative Clustering

$$\hat{y} = \beta_0 + \beta_1 x$$

Setiap titik mulai sebagai cluster sendiri, digabung bertahap.

Dendrogram: Tinggi Penggabungan Cluster



Contoh: Mengelompokkan pelanggan berdasarkan pola belanja

Metode Linkage (Jarak Antar Cluster)

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Cara mengukur jarak saat menggabungkan dua cluster.

Empat jenis linkage yang umum digunakan:

SL

Single Linkage

Jarak minimum antar titik dua cluster

rentan chaining

CL

Complete Linkage

Jarak maksimum antar titik dua cluster

compact

AL

Average Linkage

Rata-rata jarak semua pasangan titik

balanced

WD

Ward Linkage

Minimalkan varians dalam cluster (paling umum)

+ populer



```
from scipy.cluster.hierarchy import linkage
Z = linkage(X, method='ward')
print(Z) # matriks penggabungan cluster
# gunakan dendrogram(Z) untuk visualisasi
```



Implementasi K-Means dengan Scikit-Learn

API Scikit-Learn (KMeans)

Semua estimator scikit-learn mengikuti pola yang sama: Instantiate → Fit → Predict → Evaluate.

`KMeans(n_clusters=3)`

Buat objek model dengan jumlah cluster K

`.fit(X)`

Latih model: jalankan iterasi assignment & update hingga konvergen

`.predict(X_new)`

Prediksi label cluster untuk data baru

`.inertia_`

Nilai WCSS akhir (untuk metode Elbow)

`.cluster_centers_`

Koordinat centroid setiap cluster

`.labels_`

Label cluster untuk setiap data training



Implementasi K-Means & Elbow Method

```
import pandas as pd, numpy as np
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs
# make_blobs untuk data sintetis berkelompok
import matplotlib.pyplot as plt

# — 1. Load Dataset —————
# Dataset: Data pelanggan sintetis (3 kelompok)
np.random.seed(42)
n = 200
X, y_true = make_blobs(n_samples=n, centers=3,
                        cluster_std=1.2, random_state=42)

# — 2. Metode Elbow —————
wcss = []
for k in range(1, 10):

# — 3. Fit Model (K optimal dari Elbow) —————
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)

print(f"WCSS : {model.inertia_:.3f}")
print(f"Jumlah cluster: {model.n_clusters}")
# K=3 dipilih dari titik elbow pada grafik WCSS
```

```
# — 4. Predict & Evaluate —————
labels = model.predict(X)

sil = silhouette_score(X, labels)
centroids = model.cluster_centers_
# silhouettemendekati 1 = cluster terpisah baik

print(f"Silhouette Score = {sil:.3f}")
# cluster_std kecil → cluster lebih kompak
# WCSS turun tajam lalu melandai di K=3 (elbow)

# — 5. Visualisasi —————
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis')
plt.scatter(centroids[:,0], centroids[:,1],
            c='red', marker='X', s=200)
plt.xlabel('Fitur 1')
plt.ylabel('Fitur 2')
plt.legend(); plt.show()
```



Metode Elbow

WCSS/Inertia · Cara Membaca Grafik · Menentukan K Optimal



Tabel WCSS per Jumlah Cluster (K)

$$MAE = \left(\frac{1}{n}\right) \cdot \sum |y_i - \hat{y}_i|$$

K (Jumlah Cluster)	1	2	3	4	5
WCSS (Inertia)	450	210	95	82	75
Penurunan WCSS (Δ)	—	240	115	13	7

Δ terbesar: 210→95 (K=2→3) → titik elbow ada di K=3

- ✓ Sederhana diplot hanya perlu grafik WCSS vs K
- ✓ Tidak butuh label, cukup coba beberapa nilai K
- ⚠ Titik elbow kadang tidak jelas terlihat (perlu Silhouette Score sebagai pembanding)



```
wcss = []  
for k in range(1, 6):  
    wcss.append(KMeans(k).fit(X).inertia_)
```



Cara Kerja Metode Elbow

$$RMSE = \sqrt{\left(\frac{1}{n}\right) \cdot \Sigma(y_i - \hat{y}_i)^2}$$

Hubungan WCSS dan K:

WCSS(K) selalu menurun monoton saat K bertambah
Jika K = jumlah data, maka WCSS = 0

- ✓ WCSS selalu turun ketika K bertambah
- ✓ Penurunan tajam di awal, lalu melandai (titik siku/elbow)
- ⚠ Titik elbow adalah keputusan visual, bisa terasa subjektif
- ✓ Metode paling populer untuk menentukan K awal

Elbow = titik di mana penambahan cluster tidak lagi signifikan menurunkan WCSS

```
import matplotlib.pyplot as plt
plt.plot(range(1, 6), wcss, marker='o')
plt.xlabel('Jumlah Cluster (K)')
plt.ylabel('WCSS (Inertia)')
plt.title('Elbow Method')
plt.axvline(x=3, color='red', linestyle='--')
plt.show() # cari titik siku (elbow)
```



Interpretasi Grafik Elbow

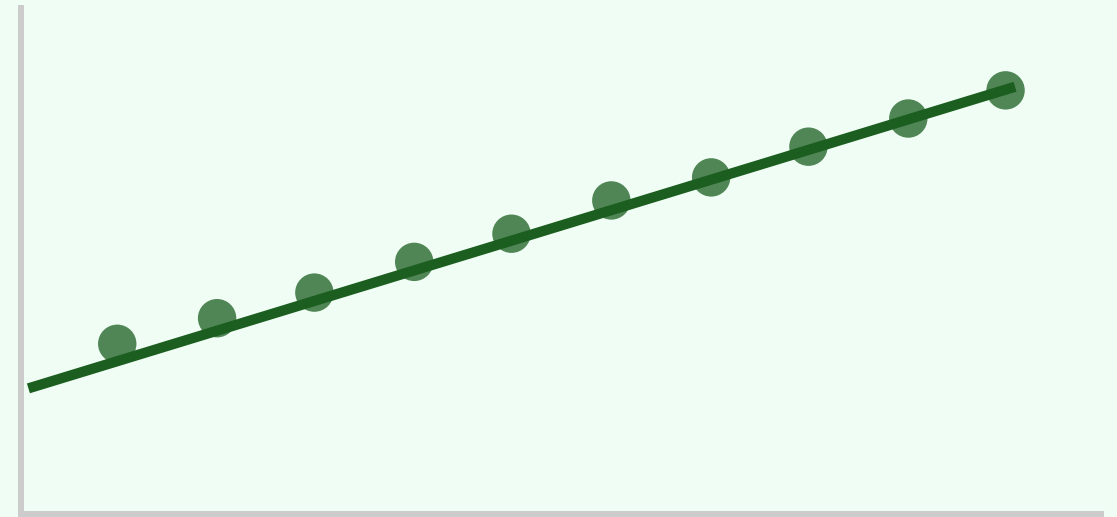
$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad ; \quad SS_{res} = \sum (y_i - \hat{y}_i)^2 \quad SS_{tot} = \sum (y_i - \bar{y})^2$$

K Terlalu Kecil (K=1)



→ Cluster terlalu general: WCSS masih sangat tinggi

K Optimal (Titik Elbow, K=3)



→ Penambahan cluster berikutnya hanya sedikit menurunkan WCSS

Interpretasi Posisi pada Kurva

Sebelum Elbow
Penurunan WCSS masih tajam, K belum cukup

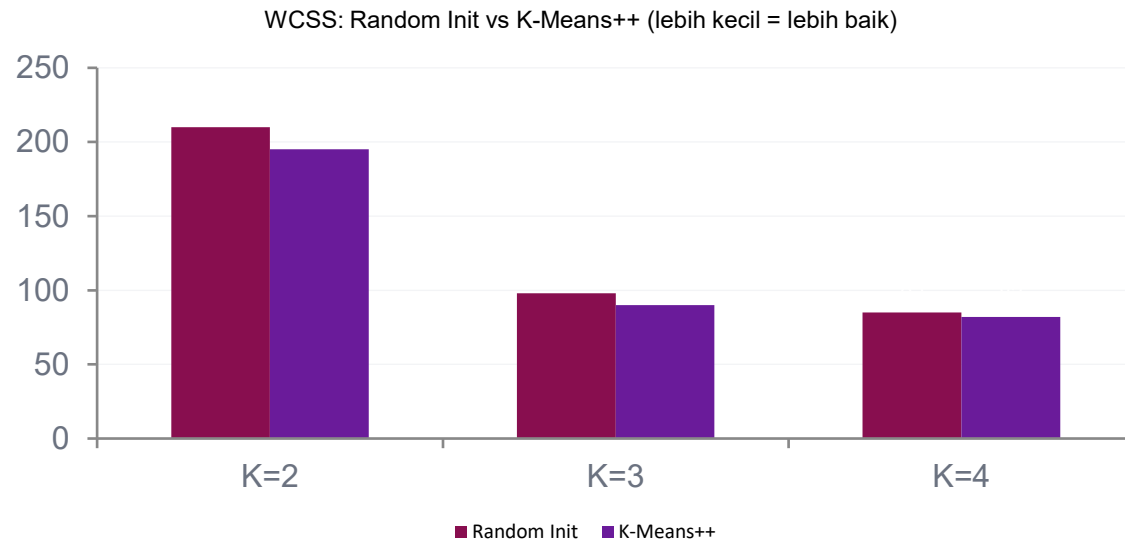
Mendekati Elbow
Penurunan mulai melandai, kandidat K optimal

Tepat di Elbow
Titik keseimbangan terbaik antara K dan WCSS

Setelah Elbow
Penurunan kecil: waspada di overfitting (cluster terlalu granular)



Inisialisasi Centroid



Cara Membaca Grafik Elbow

Plot nilai WCSS untuk beberapa K (mis. 1–10). Cari titik di mana kurva mulai melandai tajam — itulah "siku" (elbow). K-Means++ (inisialisasi cerdas) umumnya mencapai WCSS lebih rendah dari inisialisasi acak untuk K yang sama.

Gunakan parameter `init='k-means++'` (default scikit-learn) untuk hasil yang lebih stabil.



Tabel Perbandingan Algoritma Clustering

Algoritma	Kompleksitas	Keunggulan	Kelemahan	Gunakan saat
K-Means	$O(n \cdot K \cdot \text{iterasi})$	Cepat, skalabel untuk data besar	Perlu tentukan K; sensitif outlier & inisialisasi	Dataset besar, cluster bulat, K sudah diketahui
Hierarchial	$O(n^2 \log n)$	Tak perlu tentukan K; hasilkan dendrogram	Lambat untuk data besar, sulit diubah setelah digabung	Dataset kecil-menengah, butuh visualisasi hierarki
DBSCAN	$O(n \log n)$	Tahan noise/outlier; bentuk cluster bebas	Sensitif pada parameter eps & min_samples	Data dengan noise, cluster bentuk tidak beraturan
GMM	$O(n \cdot K \cdot \text{iterasi})$	Soft clustering; tangani cluster berbentuk elips	Lebih berat secara komputasi, butuh asumsi distribusi	Butuh probabilitas keanggotaan, cluster saling overlap



Aktivitas Hands-on

Lakukan aktivitas sesuai Panduan yang tertera pada Modul Pembelajaran 11



Terima Kasih